



中山大学计算机学院

人工智能

本科生实验报告

(2024 学年春季学期)

课程名称: Artificial Intelligence

教学班级	人工智能 (饶洋辉老师)	专业 (方向)	23 级地理信息科学 (辅修)
学号	23306103	姓名	赵义凯

一、实验题目

一阶逻辑归结算法

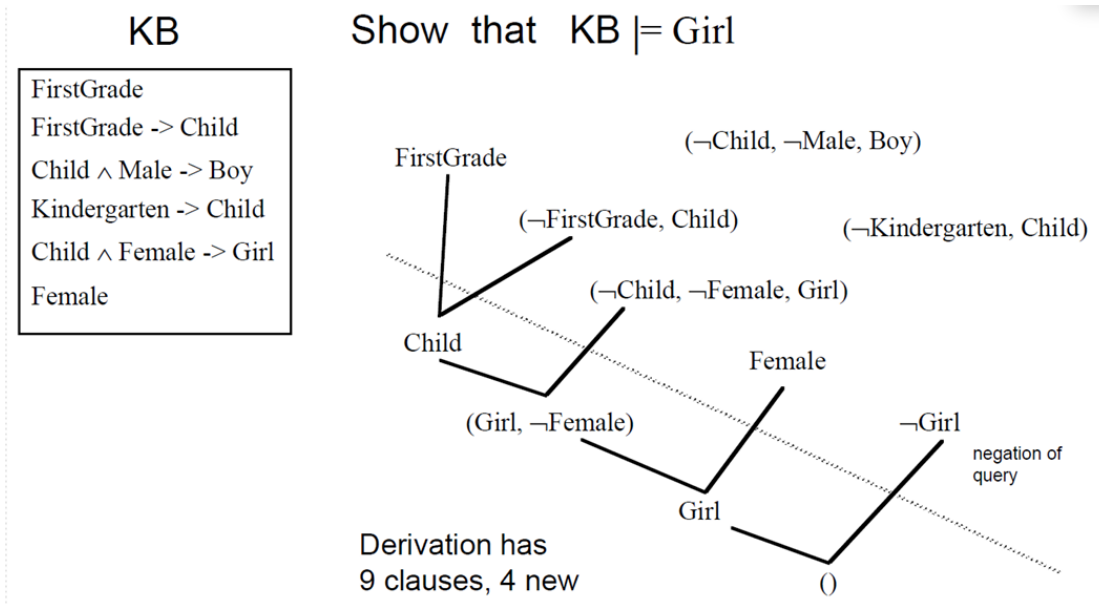
二、实验内容

1. 算法原理

1.1 一阶逻辑归结算法是基于一阶逻辑的推理方法。其核心原理是通过寻找两个子句中的互补文字(一个文字和它的否定),然后将这两个子句进行归结操作,生成新的子句。在本次实验中,我们约定了谓词、变量和常量的表示形式,子句以析取式存储。归结过程中,使用最一般合一(MGU)算法找到两个子句中互补文字的最一般合一,使得在替换变量后,互补文字能够完全匹配,从而实现归结。例如,对于子句 $P(x)$ 和 $\neg P(a)$,通过 MGU 算法找到替换 $x=a$,然后将两个子句归结为一个空子句(如果归结后只剩下空子句,则说明原知识库与查询之间存在矛盾,即查询成立)。

1.2 归结原理示意图:

图表 1 归结原理示意图

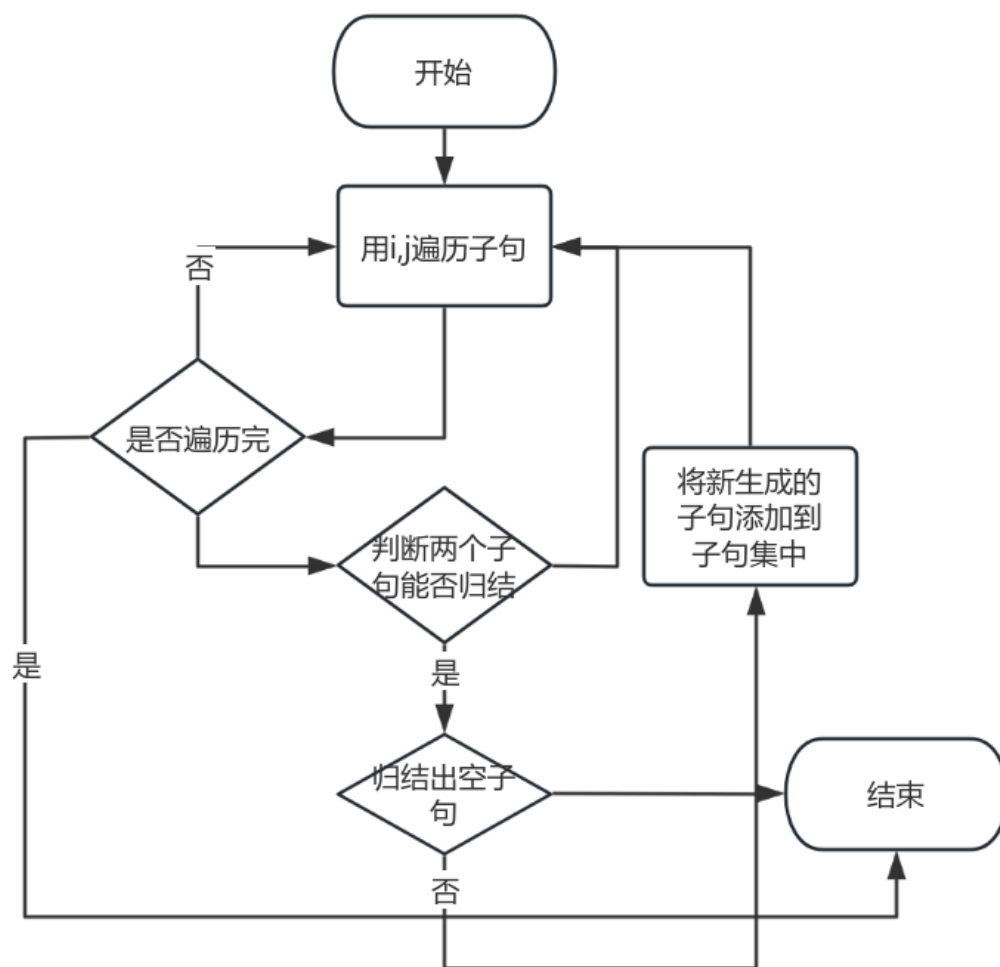




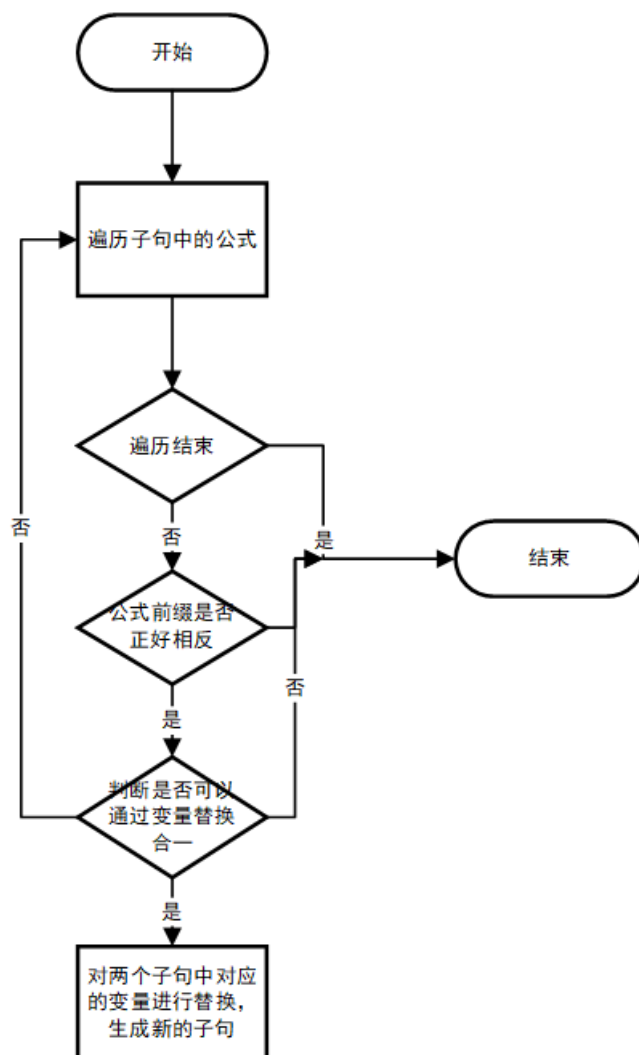
1.3 流程图展示:

输入部分:





判断两个子句能否进行归结：



2. 伪代码

Mgu 伪代码:



```
Function MGU(p, q):
    if p == q:
        return {}
    p_parts = split(p, '(')
    q_parts = split(q, '(')
    if p_parts[0] != q_parts[0]:
        return None
    p_args = split(p_parts[1][:-1], ',')
    q_args = split(q_parts[1][:-1], ',')
    subst = {}
    for i from 0 to length(p_args)-1:
        if p_args[i] != q_args[i]:
            if p_args[i][0] == 'x' and q_args[i][0] == 'x':
                subst[p_args[i]] = q_args[i]
            else:
                return None
    return subst
```

归结原理伪代码:



```
Function Resolution(KB):
    res_steps = []
    kb_copy = copy(KB)
    new_clauses = []
    while True:
        new = False
        sort(kb_copy, key=lambda x: len(x))
        for i from 0 to length(kb_copy)-1:
            for j from i + 1 to length(kb_copy)-1:
                for k from 0 to length(kb_copy[i])-1:
                    for l from 0 to length(kb_copy[j])-1:
                        if kb_copy[i][k].startswith('~') and kb_copy[i][k][1:] == kb_copy[j][l]:
                            mgu = MGU(kb_copy[i][k][1:], kb_copy[j][l])
                            if mgu:
                                new_clause = []
                                for atom in kb_copy[i]:
                                    if atom != kb_copy[i][k]:
                                        new_atom = apply_subst(atom, mgu)
                                        if new_atom not in new_clause:
                                            new_clause.append(new_atom)
                                for atom in kb_copy[j]:
                                    if atom != kb_copy[j][l]:
                                        new_atom = apply_subst(atom, mgu)
                                        if new_atom not in new_clause:
                                            new_clause.append(new_atom)
                                new_clause = sort(new_clause)
                                if new_clause not in kb_copy and new_clause not in new_clauses:
                                    new_clauses.append(new_clause)
                                    new = True
                                res_steps.append((i, k, j, l, mgu))

        if not new:
            break
        kb_copy.extend(new_clauses)
    return res_steps
```

3. 关键代码展示（带注释）



```
import re
from collections import OrderedDict
# 解析单个文字，返回谓词、参数和否定标志
def parse_literal(literal_str):
    literal_str = literal_str.strip()
    is_negated = literal_str.startswith("~")
    if is_negated:
        literal_str = literal_str[1:].strip()
    match = re.match(r"^[A-Z]\w*\(((.*)\)$", literal_str)
    if not match:
        return literal_str, (), is_negated
    predicate, args_str = match.groups()
    args = tuple(arg.strip() for arg in args_str.split(",")) if args_str else ()
    return predicate, args, is_negated
# 将文字转换为字符串形式
def literal_to_str(literal):
    predicate, args, is_negated = literal
    result = "~" if is_negated else ""
    result += f"{predicate}({','.join(args)})" if args else predicate
    return result
# 归结方法
class Sentences:
    def __init__(self, filepath):
        self.knowledge_base = []
        self.queries = []
        self.resolution_steps = []
        self.load_from_file(filepath)
    def resolution(self):
        print("归结开始:")
        print("知识库:")
        for idx, clause in enumerate(self.knowledge_base, start=1):
            print(f"{idx}: {clause_to_str(clause)}")
        print("查询 (取反形式):")
        for clause in self.queries:
            print(clause_to_str(clause))
        print()
        kb_list = sorted(self.knowledge_base, key=len)
        seen_clauses = {frozenset(clause) for clause in kb_list}
        step_counter = 0
        new_clauses_added = True
        while new_clauses_added:
            new_clauses_added = False
            for i in range(len(kb_list)):
                for j in range(i + 1, len(kb_list)):
                    new_resolvents = self.resolve(kb_list[i], kb_list[j])
                    for new_clause, substitution in new_resolvents:
                        step_counter += 1
                        step_log = f"[步骤 {step_counter}] 归结: {clause_to_str(kb_list[i])} 与 {clause_to_str(kb_list[j])} =>"
                        print(step_log)
                        self.resolution_steps.append(step_log)
                        if not new_clause:
                            print("得到空子句 (NIL), 归结成功!")
                            return
                        frozen_clause = frozenset(new_clause)
                        if frozen_clause not in seen_clauses:
                            seen_clauses.add(frozen_clause)
                            kb_list.append(new_clause)
                            kb_list = sorted(kb_list, key=len)
                            new_clauses_added = True
            if not new_clauses_added:
                print("归结失败, 无法推出空子句。")
                return
```

4. 创新点&优化（如果有）

在代码实现过程中，为提升归结效率，采用了优先选取长度短的子句进行归结的优化策略。在 resolution 方法里，对知识库中的子句按长度排序，每次添加新子句后重新排序。这样在归结时优先处理短子句，由于短子句文字数量少，寻找互补文字和进行合一操作的计算复杂度较低，减少了不必要的计算量。

三、实验结果及分析

1. 实验结果展示示例（可图可表可文字，尽量可视化）

优化前：

```
2: (~GradStudent(x), Student(x))
3: (~Student(x), HardWorker(x))
4: (~HardWorker(sue))
QUERY (negated form):
(~HardWorker(sue))

[Step 1] 归结: (GradStudent(sue)) 与 (~GradStudent(x), Student(x)) => (Student(sue)), 使用MGU: {'x': 'sue'}
[Step 2] 归结: (~GradStudent(x), Student(x)) 与 (~Student(x), HardWorker(x)) => (~GradStudent(x), HardWorker(x)), 使用MGU: {}
[Step 3] 归结: (~Student(x), HardWorker(x)) 与 (~HardWorker(sue)) => (~Student(sue)), 使用MGU: {'x': 'sue'}
[Step 4] 归结: (GradStudent(sue)) 与 (~GradStudent(x), Student(x)) => (Student(sue)), 使用MGU: {'x': 'sue'}
[Step 5] 归结: (GradStudent(sue)) 与 (~GradStudent(x), HardWorker(x)) => (HardWorker(sue)), 使用MGU: {'x': 'sue'}
[Step 6] 归结: (~GradStudent(x), Student(x)) 与 (~Student(x), HardWorker(x)) => (~GradStudent(x), HardWorker(x)), 使用MGU: {}
[Step 7] 归结: (~GradStudent(x), Student(x)) 与 (~Student(sue)) => (~GradStudent(sue)), 使用MGU: {'x': 'sue'}
[Step 8] 归结: (~Student(x), HardWorker(x)) 与 (~HardWorker(sue)) => (~Student(sue)), 使用MGU: {'x': 'sue'}
[Step 9] 归结: (~Student(x), HardWorker(x)) 与 (Student(sue)) => (HardWorker(sue)), 使用MGU: {'x': 'sue'}
[Step 10] 归结: (~HardWorker(sue)) 与 (~GradStudent(x), HardWorker(x)) => (~GradStudent(sue)), 使用MGU: {'x': 'sue'}
[Step 11] 归结: (Student(sue)) 与 (~Student(sue)) => NIL, 使用MGU: {}
推出空子句 (NIL), 归结成功!
```

优化后：

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

1: 显示案例内容
2: 输出归结过程
0: 返回, 重新选择案例
选择你想实现的功能: 2
归结开始:
知识库:
1: (GradStudent(sue))
2: (~GradStudent(x), Student(x))
3: (~Student(x), HardWorker(x))
4: (~HardWorker(sue))
查询 (取反形式):
': 'sue'}
[步骤 2] 归结: (GradStudent(sue)) 与 (~GradStudent(x), Student(x)) => (Student(sue)), 合一: {'x': 'sue'}
[步骤 3] 归结: (~HardWorker(sue)) 与 (~Student(x), HardWorker(x)) => (~Student(sue)), 合一: {'x': 'sue'}
[步骤 4] 归结: (Student(sue)) 与 (~Student(sue)) => NIL, 合一: {}
得到空子句 (NIL), 归结成功!
Traceback (most recent call last):
  File "d:\大二下人工智能资料\实验课\新建文件夹\ai class\main.py", line 55, in <module>
    test1.reindex()
```

2. 评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

四、思考题

1. 如果知识库中存在大量子句，如何进一步优化归结算法以提高效率？
可以把相似的子句放在一起，这样找互补文字的时候就能更快找到。还能定期清理那些对归结没什么用的子句，让知识库变小，归结的时候计算量也就少了。
2. 在实际应用场景中，一阶逻辑归结算法可能会遇到哪些挑战？



实际生活中的知识很复杂，有些知识很难用我们规定的这种一阶逻辑的方式表示出来。而且知识还可能经常变化，新的知识加进来后，用原来的算法归结可能会变慢或者出问题。

五、 参考资料

- [1] 【人工智能导论】实验二 一阶逻辑归结算法 - 哔哩哔哩 (bilibili.com)
- [2] 理论第 2 讲 知识表示和推理.pdf